

Solution Requirements

Date	5 th July 2026
Team ID	RAHMA HUSSEIN JUMA
Project Name	CREDIT CARD APPROVAL PREDICTION
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	The current version does not require login, since analysts/customers just submit a form. If extended, users would log in with an email and password, matching the Users entity in our ERD (UserID, Email, Password).	Low
2	Authorization levels	Three user roles map to the Role field in the Users entity: Analyst (reviews and submits applications), Compliance Officer (batch-screens high-risk applicants), Customer (self-service eligibility check). Each would see a different view of the same underlying prediction engine.	Medium

3	External interfaces	The system connects to the IBM Watson Machine Learning API for optional cloud-hosted model deployment. The dataset itself (application_record.csv, credit_record.csv) was sourced externally from Kaggle, not from a live third-party API.	Medium
4	Transactions processing	When a user submits the form, the Flask app collects the input, applies the same preprocessing used in training (encoding, scaling), passes it to the saved model, and returns an Approved/Rejected result within seconds. No payment transactions are processed — this is a decision-support system, not a payment system.	High
5	Reporting	The system generates a model comparison report (model_comparison_results.csv) showing accuracy, precision, recall, F1, and ROC-AUC for all four trained models, plus exploratory charts (income distribution, correlation heatmap, etc.) generated during the EDA phase.	Medium
6	Business rules, etc.	Applicants with a payment status of 60+ days overdue (STATUS codes 2-5) in their credit history are labeled as high-risk and predicted for rejection; all other applicants are labeled safe. Because risky applicants are rare (~1.7% of the dataset), models are trained with class_weight="balanced" to	High

		avoid ignoring the minority class.	
7	Compliance to laws or regulations	Since this system makes automated credit decisions using demographic fields (gender, family status), it would need to comply with fair lending regulations (e.g., Equal Credit Opportunity Act) to avoid discriminatory bias, and data privacy regulations (e.g., GDPR) for handling applicants' personal and financial information.	High
8	Other	The system should log every prediction made (applicant input, model used, result, timestamp) for audit purposes, matching the Approval_Prediction entity in the ERD (PredictionID, ApplicantID, ModelID, ApprovalResult, PredictionDate).	Medium

Step 2: Non-Functional Requirements (NFR)

1	Performance & Speed	The system must return an approval/rejection prediction almost immediately after form submission, since the model is pre-loaded rather than retrained per request.	Prediction returned in under 2 seconds
---	---------------------	--	--

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
------	--------------	-------------------------	-------------------------------------

2	Scalability	The current single Flask instance handles individual predictions; if demand grows, the model can be hosted on IBM Watson Machine Learning to serve many requests at once.	Supports at least 100 concurrent prediction requests when cloud-hosted
3	Security & Data Privacy	Applicant financial and personal data (income, employment, family details) must be protected in transit and at rest, especially since demographic fields are involved.	Data transmitted over HTTPS; no plaintext storage of sensitive fields
4	Reliability & Availability	The trained model, scaler, and column list are saved as reusable files, so the app can restart without needing to retrain, keeping predictions consistently available.	99% uptime for the web application
5	Usability & Accessibility	The form must be usable by both technical (analyst/compliance officer) and non-technical (customer) users, with clear labels and simple dropdowns for categorical fields.	A first-time user can complete the form without external instructions
6	<i>Other</i>	Maintainability: the pipeline is split into clearly separated stages (data collection, preprocessing, model training, application), so any one stage can be updated independently. Portability: built with standard Python libraries, runnable on any machine with requirements.txt installed.	Any single stage (e.g. swapping the model) can be updated without modifying the others